14. Merge Conflict Kabusunu AI ile 60 Saniyede Çözmek

<<<<<<< HEAD ve >>>>>>> Çatışmalarını Korkusuzca Yönetmek
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Junior mühendisler neden merge conflict çıktığında panikleyip branch'i silme noktasına gelir ve AI her iki tarafın niyetini nasıl birleştirebilir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Merge Conflict bir hata değil; iki geliştiricinin aynı kod bloğunu aynı anda değiştirdiğini bildiren bir işbirliği sinyalidir. - 📌 Çözüm 'benimkini al' ya da 'onunkini al' ikileminde değil; her iki tarafın iş mantığını harmanlamaktadır. - 📌 AI modelleri, conflict marker'larını (`<<<<<<<` , `=======` , `>>>>>>>`) okuyup her iki mantığı kusursuzca sentezleyebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İki ressamın aynı tuval üzerinde çalıştığını hayal edin.

Ahmet tuvalin sağ köşesine güzel bir çınar ağacı çiziyor. Aynı anda Zeynep de tuvalin tam o köşesine bir göl ve kuğu çiziyor. Akşam olup resimleri birleştirmeye kalktıklarında boyalar birbirine giriyor!

İşte bu bir **Merge Conflict**'tir. Git boyaları karıştırmak istemediği için fırçayı elinizden alır ve sorar: *"Burada hem ağaç hem kuğu mu olacak, yoksa birinden vaz mı geçeceksiniz?"*

Junior yazılımcılar genellikle panikleyip rastgele tuşlara basar veya başkasının kodunu siler. Oysa yapay zeka harika bir sanat eleştirmeni gibi bakar: *"Gölün kıyısına ağacı yerleştirelim, ikisi de kalsın"* diyerek çatışmayı çözer.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Merge Conflict Anatomisi:

```
<<<<<<< HEAD (Senin Değişikliğin - Current Change)
const taxRate = 0.20; // Türkiye KDV güncellemesi
=======
const taxRate = 0.18;
const hasCustomDiscount = true; // Takım arkadaşının eklediği yeni özellik
>>>>>>> incoming-branch (Gelen Değişiklik)

_____________________________________________________

AI Sentezi (Doğru Çözüm):
const taxRate = 0.20; // Güncel oran korundu
const hasCustomDiscount = true; // Arkadaşının özelliği dahil edildi
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Git Merge ve Çatışma Çözüm Uzmanı
Aşağıdaki dosyada bir Git Merge Conflict meydana geldi.

## ÇATIŞAN KOD BLOĞU:
```typescript
<<<<<<< HEAD
export async function fetchUserOrders(userId: string) {
  // Sıralama özelliği eklendi
  return await db.orders.findMany({ where: { userId }, orderBy: { createdAt:
'desc' } });
}
=======
export async function fetchUserOrders(userId: string, limit: number = 10) {
  // Sayfalama (pagination) limiti eklendi
  return await db.orders.findMany({ where: { userId }, take: limit });
}
>>>>>>> feature/pagination
````

GÖREV:

1. Her iki tarafın ne amaçladığını ayrı ayrı açıkla.
2. Her iki özelliği de (hem orderBy hem take limiti) içeren birleşik ve hatasız nihai fonksiyonu yaz.
3. Conflict marker'larını tamamen temizle.
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü / Aceleci Çözüm: Takım arkadaşının limit parametresini silip sadece kendi
kodunu bırakmak
export async function fetchUserOrders(userId: string) {
  // Arkadaşının yaptığı tüm sayfalama mantığı çöpe gitti!
  return await db.orders.findMany({ where: { userId }, orderBy: { createdAt:
'desc' } });
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
// AI ile Sentezlenmiş Doğru Çözüm: Her iki özellik korunur
export interface FetchOrdersOptions {
  limit?: number;
  direction?: 'asc' | 'desc';
}

export async function fetchUserOrders(
  userId: string,
  options: FetchOrdersOptions = {}
) {
  const { limit = 10, direction = 'desc' } = options;

  return await db.orders.findMany({
    where: { userId },
    orderBy: { createdAt: direction },
    take: limit
  });
}
```

💡 **Kod Analizi:** Çatışma çözülürken tek bir tarafın kodunu ezmek yerine, fonksiyon bir seçenekler objesi (`FetchOrdersOptions`) alacak şekilde genişletilmiş; böylece hem sıralama hem de sayfalama özellikleri kusursuzca birleştirilmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Panikleyip `git checkout --ours` veya `git checkout --theirs` diyerek bir tarafı tamamen ezmek.
  - **Çözüm:** Her zaman iki tarafın kodunu da dikkatle okuyup birleştirin.
- **Tuzak:** Conflict marker'larını (`<<<<<<<` , `=======`) dosyada unutup commit etmek.
```

- **Çözüm:** Commit etmeden önce mutlaka `git diff` ve testleri (`npm test`) çalıştırın.
- **Tuzak:** Haftalarca ana branch'ten (`main`) pull almayıp çatışmaların devasa bir boyuta ulaşmasına izin vermek.
  - **Çözüm:** Her sabah çalışmaya başlarken `git pull origin main` ile yerel branch'inizi güncel tutun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 14: Korkusuz Git - Merge Conflict'leri AI ile Çözmek*

- 🖥️ **Slayt 1:** Merge Conflict nedir ve neden korkulmamalıdır?
- 🖥️ **Slayt 2:** HEAD vs Incoming: Çatışma etiketlerini okumak.
- 🖥️ **Slayt 3:** Cursor ve VS Code 3-Way Merge Editörünü kullanmak.
- 🖥️ **Slayt 4:** AI ile çatışan iki iş mantığını tek bir fonksiyonda sentezlemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Lokalde iki farklı branch açın. Aynı dosyanın 10. satırını iki branch'te farklı değiştirip merge çatışması üretin. IDE'nin conflict arayüzünü ve AI 'Resolve Conflict' butonunu kullanarak çatışmayı 30 saniyede çözün ve testleri çalıştırın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bir test reposunda bilerek bir merge conflict yaratın. Conflict etiketlerini AI'a vererek çözdürmeyi deneyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Git merge conflict çıktığında `<<<<<<< HEAD` etiketi neyi temsil eder? - A) Uzaktaki sunucuda silinen kodları - B) Şu an üzerinde bulunduğunuz mevcut branch'teki yerel değişiklikleri - C) GitHub şifrenizi - D) Hatalı yazılmış CSS kodunu > **Doğru Yanıt:** B > **Açıklama:** `HEAD` , şu an aktif olarak bulunduğunuz branch'teki (mevcut) değişiklikleri simgeler.

Soru 2: Büyük ve sancılı merge conflict'lerini önlemenin en etkili takım alışkanlığı nedir? - A) Hiç kod yazmamak - B) Küçük branch'lerle çalışmak ve her gün ana branch'ten (`main`) güncellemeleri çekmek (Pull/Rebase) - C) Sadece ayda bir commit atmak - D) Tüm ekibin aynı bilgisayardan kod yazması > **Doğru Yanıt:** B > **Açıklama:** Sık aralıklarla ana branch ile senkronize olmak ve küçük branch'ler kullanmak çatışmaların büyümesini engeller.